

Chapitre 3 : Créer et manipuler les données en SQL

Par **Robert DIASSÉ**, Ingénieur Logiciel et Data

> **Cours** : Base de données 1, fondements, modélisation et SQL · **Fil rouge** : la clinique Santé Plus · **Version** : 2026-10-31

Fiche du chapitre

Objectifs (OS3)	A la fin du chapitre, l'apprenant sait : 1) choisir le type de données de chaque colonne ; 2) traduire un MLD en `CREATE TABLE` avec clés primaires, clés étrangères, `NOT NULL`, `UNIQUE`, `CHECK` et `DEFAULT` ; 3) modifier et supprimer une structure (`ALTER`, `DROP`, `TRUNCATE`) ; 4) insérer, modifier et supprimer des lignes (`INSERT`, `UPDATE`, `DELETE`) et « insérer ou mettre à jour » (`MERGE`) ; 5) choisir le comportement d'une clé étrangère à la suppression ; 6) numéroter automatiquement (identité, séquence) ; 7) regrouper des modifications dans une transaction (`COMMIT`, `ROLLBACK`, `SAVEPOINT`).
Prérequis	Chapitres 1 et 2 ; atelier 2 terminé (votre MLD de la clinique)
Durée	🟢 Débutant : 12 h · 🔵 Standard : 9 h · 🟣 Expert : 6 h (atelier compris)
Matériel	Votre SGBD (MySQL, PostgreSQL ou Oracle), la base `clinique`
Livrable	L'atelier 3 : le script complet de création et de remplissage de la base de la clinique

Comment lire ce chapitre

- **Noyau** : obligatoire pour tous les niveaux.
- 🟢 **Débutant** : explications pas à pas. A sauter si le public est à l'aise.
- 🔵 **Standard** : le niveau de référence.
- 🟣 **Expert** : approfondissements.

Le SQL se divise en familles. Ce chapitre couvre les trois premières ; le chapitre 4 sera consacré à la lecture (`SELECT`).

Famille	Nom complet	Instructions	Section
LDD	Langage de Définition des Données	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`	1 à 4

Famille	Nom complet	Instructions	Section
LMD	Langage de Manipulation des Données	`INSERT`, `UPDATE`, `DELETE`, `MERGE` (et `SELECT`, chapitre 4)	5 et 6
LCT	Langage de Contrôle des Transactions	`COMMIT`, `ROLLBACK`, `SAVEPOINT`	8

Dans votre base `clinique` (ou votre schéma Oracle), commencez par **supprimer** les tables de l'atelier 1 : `DROP TABLE medecin;` et, si vous l'avez créée, `DROP TABLE patient;`. Ce chapitre recrée la base proprement, à partir du MLD du chapitre 2.

---

1. Les types de données

Chaque colonne a un **type**, qui fixe ce qu'elle peut contenir. Le SGBD refuse toute valeur qui ne convient pas (vous l'avez vu avec la date du 30 février).

Contenu	Type standard (MySQL et PostgreSQL)	🔶 Oracle	Exemple à la clinique
Nombre entier	`INT` (ou `INTEGER`), `SMALLINT`	`NUMBER(10)` (`INT` accepté)	`num_patient`, `etage`, `tarif` en FCFA
Nombre décimal exact	`DECIMAL(p, s)` (ou `NUMERIC`) : `p` chiffres, dont `s` après la virgule	`NUMBER(p, s)`	`poids DECIMAL(5, 2)` : jusqu'à 999,99
Texte de longueur variable	`VARCHAR(n)`	`VARCHAR2(n)`	`nom_patient VARCHAR(50)`
Texte de longueur fixe	`CHAR(n)`	`CHAR(n)`	`sexe CHAR(1)`
Texte long	`TEXT`	`CLOB`	Un compte rendu médical
Date	`DATE`	`DATE` (contient aussi l'heure)	`date_naissance`
Date et heure	`TIMESTAMP` (MySQL : aussi `DATETIME`)	`TIMESTAMP`	Heure d'arrivée aux urgences
Vrai / faux	`BOOLEAN`	`BOOLEAN` (Oracle 23 et plus) ; avant : `NUMBER(1)`	`est_assure`

> 🟢 **Débutant : bien choisir**
>
> - Un **identifiant** numérique : `INT`. Un **code** (`PED`, `PARA500`) : `VARCHAR`.
> - Un **montant en FCFA** : `INT` (pas de centimes). Une valeur avec des décimales exactes (un poids, un taux) : `DECIMAL`. Évitez `FLOAT` pour l'argent : il arrondit.
> - Un **téléphone**, un **code postal**, un **matricule** : `VARCHAR`, car on ne calcule pas avec, et le `0` en tête ou le `+221` seraient perdus dans un nombre.
> - `VARCHAR(n)` : prévoyez large mais réaliste (50 pour un nom).

2. Créer les tables et leurs contraintes

2.1 Le script de création de la clinique

Voici le MLD du chapitre 2 (section 5.2), traduit en SQL. Exécutez-le **en entier**, dans cet ordre :

```sql
CREATE TABLE service (
    code_service VARCHAR(5)  PRIMARY KEY,
    nom_service  VARCHAR(40) NOT NULL UNIQUE,
    etage        INT         NOT NULL CHECK (etage BETWEEN 0 AND 5)
);

CREATE TABLE medecin (
    num_medecin    INT         PRIMARY KEY,
    nom_medecin    VARCHAR(50) NOT NULL,
    prenom_medecin VARCHAR(50) NOT NULL,
    specialite     VARCHAR(40) NOT NULL,
    tel_medecin    VARCHAR(20),
    code_service   VARCHAR(5)  NOT NULL,
    CONSTRAINT fk_medecin_service FOREIGN KEY (code_service) REFERENCES service (code_service)
);

CREATE TABLE patient (
    num_patient    INT         PRIMARY KEY,
    nom_patient    VARCHAR(50) NOT NULL,
    prenom_patient VARCHAR(50) NOT NULL,
    date_naissance DATE,
    sexe           CHAR(1)     CHECK (sexe IN ('F', 'M')),
    tel_patient    VARCHAR(20),
    ville          VARCHAR(40) DEFAULT 'Dakar'
);

CREATE TABLE consultation (
    num_consultation  INT          PRIMARY KEY,
    date_consultation DATE         NOT NULL,
    motif             VARCHAR(100),
    tarif             INT          NOT NULL CHECK (tarif >= 0),
    num_patient       INT          NOT NULL,
    num_medecin       INT          NOT NULL,
    CONSTRAINT fk_consultation_patient FOREIGN KEY (num_patient) REFERENCES patient (num_patient),
    CONSTRAINT fk_consultation_medecin FOREIGN KEY (num_medecin) REFERENCES medecin (num_medecin)
);

CREATE TABLE medicament (
    code_medicament VARCHAR(10) PRIMARY KEY,
    nom_medicament  VARCHAR(60) NOT NULL,
    forme           VARCHAR(20)
);

CREATE TABLE prescription (
    num_consultation INT          NOT NULL,
    code_medicament  VARCHAR(10)  NOT NULL,
    posologie        VARCHAR(100) NOT NULL,
    duree_jours      INT          CHECK (duree_jours > 0),
    CONSTRAINT pk_prescription PRIMARY KEY (num_consultation, code_medicament),
    CONSTRAINT fk_prescription_consultation FOREIGN KEY (num_consultation) REFERENCES consultation
        (num_consultation),
    CONSTRAINT fk_prescription_medicament FOREIGN KEY (code_medicament) REFERENCES medicament
        (code_medicament)
);
```

2.2 Les contraintes, une par une

Contrainte	Ce qu'elle impose	Dans le script
`PRIMARY KEY`	Valeur **unique** et **obligatoire** : identifie chaque ligne (intégrité d'entité)	`code_service VARCHAR(5) PRIMARY KEY`
`PRIMARY KEY (a, b)`	Clé primaire **composée** : c'est le **couple** qui est unique	`pk_prescription`
`NOT NULL`	Valeur obligatoire	`nom_service ... NOT NULL`
`UNIQUE`	Pas deux fois la même valeur (mais ce n'est pas l'identifiant)	Deux services ne portent pas le même nom
`CHECK (condition)`	La valeur doit respecter la condition (intégrité de domaine)	`etage BETWEEN 0 AND 5`, `sexe IN ('F', 'M')`
`DEFAULT valeur`	Valeur mise automatiquement si l'on n'en donne pas	`ville ... DEFAULT 'Dakar'`
`FOREIGN KEY (col) REFERENCES table (col)`	La valeur doit **exister** dans la table désignée (intégrité référentielle)	Un médecin est dans un service qui existe
`CONSTRAINT nom ...`	Donne un **nom** à la contrainte, qui apparaîtra dans les messages d'erreur et permettra de la supprimer	`fk_medecin_service`

Morceau	Ce qu'il veut dire
L'ordre des `CREATE TABLE`	Une table **parent** (`service`) doit exister avant la table **enfant** qui la désigne (`medecin`). On crée donc : `service`, `medecin`, `patient`, `consultation`, `medicament`, `prescription`
`code_service VARCHAR(5) NOT NULL` dans `medecin`	La clé étrangère a **le même type** que la clé primaire qu'elle désigne. `NOT NULL` car la cardinalité est `1,1` (chapitre 2, section 5.3)
`CHECK (sexe IN ('F', 'M'))` sans `NOT NULL`	Le sexe peut être inconnu (`NULL`) ; s'il est donné, ce sera `F` ou `M`
Contraintes écrites **à la fin** de la table	Obligatoire pour une clé primaire composée ; plus lisible pour les clés étrangères

> 🟢 **Débutant : comment nommer les contraintes ?** Une convention simple : `pk_` + table pour une clé primaire, `fk_` + table enfant + `_` + table parent pour une clé étrangère, `ck_` + colonne pour un `CHECK`. Quand une erreur cite `fk_consultation_patient`, vous savez tout de suite où chercher.

> 🧨 **Essayez** : supprimez toutes les tables (dans l'ordre inverse : `DROP TABLE prescription;` … `DROP TABLE service;`), puis créez `medecin` **avant** `service`.
>
> - MySQL (MariaDB) : `ERROR 1005 (HY000): Can't create table 'clinique'.'medecin' (errno: 150 "Foreign key constraint is incorrectly formed")`
> - PostgreSQL : `ERREUR: la relation « service » n'existe pas`
>
> La clé étrangère désigne une table qui n'existe pas encore. Recréez tout dans le bon ordre.

> 🔶 **Oracle** Le script fonctionne tel quel sur Oracle. Pour suivre les usages d'Oracle, remplacez `VARCHAR` par `VARCHAR2` et `INT` par `NUMBER(10)`. Oracle ajoute une règle : un nom de contrainte (et de table) est unique dans tout le schéma ; gardez donc des noms précis comme `fk_consultation_patient`.

3. Remplir les tables : `INSERT`

3.1 Deux façons d'écrire un `INSERT`

```
INSERT INTO service (code_service, nom_service, etage) VALUES ('GEN', 'Consultations générales',
        0);
INSERT INTO service (code_service, nom_service, etage) VALUES ('PED', 'Pédiatrie', 1);
INSERT INTO service (code_service, nom_service, etage) VALUES ('CAR', 'Cardiologie', 2);

INSERT INTO medecin VALUES (1, 'Ba', 'Mamadou', 'Médecine générale', '77 100 20 30', 'GEN');
INSERT INTO medecin VALUES (2, 'Faye', 'Aminata', 'Pédiatrie', '78 200 30 40', 'PED');
INSERT INTO medecin VALUES (3, 'Sow', 'Ibrahima', 'Cardiologie', NULL, 'CAR');
INSERT INTO medecin VALUES (4, 'Kane', 'Ndeye', 'Pédiatrie', '76 300 40 50', 'PED');

INSERT INTO patient (num_patient, nom_patient, prenom_patient, date_naissance, sexe, tel_patient,
        ville)
VALUES (1, 'Diop', 'Awa', DATE '1998-04-12', 'F', '77 111 22 33', 'Dakar');
INSERT INTO patient (num_patient, nom_patient, prenom_patient, date_naissance, sexe, tel_patient,
        ville)
VALUES (2, 'Ndiaye', 'Moussa', DATE '1975-11-03', 'M', '70 444 55 66', 'Thiès');
INSERT INTO patient (num_patient, nom_patient, prenom_patient, date_naissance, sexe, tel_patient)
VALUES (3, 'Sarr', 'Fatou', DATE '2019-06-25', 'F', '76 777 88 99');

INSERT INTO consultation VALUES (1, DATE '2026-10-05', 'Fièvre', 10000, 1, 1);
INSERT INTO consultation VALUES (2, DATE '2026-10-05', 'Toux persistante', 15000, 3, 2);
INSERT INTO consultation VALUES (3, DATE '2026-10-12', 'Contrôle tension', 20000, 2, 3);

INSERT INTO medicament VALUES ('PARA500', 'Paracétamol 500 mg', 'comprimé');
INSERT INTO medicament VALUES ('AMOX250', 'Amoxicilline 250 mg', 'sirop');

INSERT INTO prescription VALUES (1, 'PARA500', '1 comprimé 3 fois par jour', 5);
INSERT INTO prescription VALUES (2, 'AMOX250', '1 cuillère matin et soir', 7);
INSERT INTO prescription VALUES (2, 'PARA500', '1/2 comprimé si fièvre', 3);

SELECT num_patient, nom_patient, ville FROM patient;
```

```
 num_patient | nom_patient | ville
-------------+-------------+-------
           1 | Diop        | Dakar
           2 | Ndiaye      | Thiès
           3 | Sarr        | Dakar
(3 lignes)
```

Morceau	Ce qu'il veut dire
`INSERT INTO service (code_service, nom_service, etage) VALUES (...)`	Avec la **liste des colonnes** : les valeurs suivent cet ordre-là. C'est la forme **recommandée** : elle reste juste si l'on ajoute une colonne à la table
`INSERT INTO medecin VALUES (...)`	Sans liste : il faut une valeur pour **chaque** colonne, dans l'ordre du `CREATE TABLE`
Fatou Sarr, sans `ville`	La colonne n'est pas citée : elle reçoit sa valeur par **défaut**, `'Dakar'`
L'ordre des `INSERT`	Comme pour les tables : un service avant ses médecins, un patient et un médecin avant leur consultation
`'1 cuillère matin et soir'`	Un texte entre apostrophes. Pour une apostrophe **dans** le texte, on la **double** : `'Contes d''Amadou Koumba'`

> 🔵 **Standard : plusieurs lignes en un `INSERT`** MySQL et PostgreSQL acceptent plusieurs lignes à la fois : `INSERT INTO medicament VALUES ('IBU400', 'Ibuprofène 400 mg', 'comprimé'), ('SRO', 'Sels de réhydratation', 'sachet');` . 🔶 Oracle ne l'accepte qu'à partir de la version 23 ; avant, on écrit un `INSERT` par ligne.

3.2 Les contraintes au travail

> 🧨 **Essayez** : `INSERT INTO consultation VALUES (4, DATE '2026-10-20', 'Douleur', 10000, 99, 1);` (le patient n°99 n'existe pas).
>
> - MySQL : `ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (... CONSTRAINT 'fk_consultation_patient' ...)`
> - PostgreSQL : `ERREUR: une instruction insert ou update sur la table « consultation » viole la contrainte de clé étrangère « fk_consultation_patient »` puis `DÉTAIL : La clé (num_patient)=(99) n'est pas présente dans la table « patient ».`
>
> Traduction de MySQL : « impossible d'ajouter une ligne **enfant** : une contrainte de clé étrangère échoue ». Le nom de la contrainte vous dit laquelle.

> 🧨 **Essayez** : `INSERT INTO patient (num_patient, nom_patient, prenom_patient, sexe) VALUES (4, 'Fall', 'Ibou', 'X');`
>
> - MySQL (MariaDB) : `ERROR 4025 (23000): CONSTRAINT 'patient.sexe' failed for 'clinique'.'patient'`
> - PostgreSQL : `ERREUR: la nouvelle ligne de la relation « patient » viole la contrainte de vérification « patient_sexe_check »`
>
> Le `CHECK` refuse `X`. Les noms `patient.sexe` et `patient_sexe_check` ont été **inventés par le SGBD**, car la contrainte n'avait pas de nom. (MySQL 8 affiche un message un peu différent de MariaDB, avec le même sens.)

> 🧨 **Essayez** : `INSERT INTO service VALUES ('PED2', 'Pédiatrie', 1);`
>
> - MySQL : `ERROR 1062 (23000): Duplicate entry 'Pédiatrie' for key 'nom_service'`
> - PostgreSQL : `ERREUR: la valeur d'une clé dupliquée rompt la contrainte unique « service_nom_service_key »`
>
> Le code `PED2` est nouveau, mais le **nom** existe déjà : c'est `UNIQUE` qui refuse.

> 🧨 **Essayez** : `INSERT INTO prescription VALUES (1, 'PARA500', 'encore', 2);`
>
> - MySQL : `ERROR 1062 (23000): Duplicate entry '1-PARA500' for key 'PRIMARY'`
> - PostgreSQL : `ERREUR: la valeur d'une clé dupliquée rompt la contrainte unique « pk_prescription »`
>
> La clé primaire composée interdit de prescrire deux fois le même médicament dans la même consultation.

4. Modifier la structure : `ALTER`, `DROP`, `TRUNCATE`

4.1 `ALTER TABLE`

La clinique veut enregistrer le **groupe sanguin** des patients :

```
ALTER TABLE patient ADD groupe_sanguin VARCHAR(3);
ALTER TABLE patient ADD CONSTRAINT ck_groupe
    CHECK (groupe_sanguin IN ('A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-'));
```

Instruction	Ce qu'elle fait
`ALTER TABLE patient ADD groupe_sanguin VARCHAR(3);`	Ajoute une colonne. Les lignes existantes y reçoivent `NULL` (ou la valeur `DEFAULT` si l'on en donne une)

Instruction	Ce qu'elle fait
`ALTER TABLE patient ADD CONSTRAINT ck_groupe CHECK (...);`	Ajoute une contrainte à une table existante. Elle est **refusée** si des lignes existantes ne la respectent pas
`ALTER TABLE patient DROP COLUMN groupe_sanguin;`	Supprime une colonne **et toutes ses valeurs**
`ALTER TABLE prescription DROP CONSTRAINT fk_prescription_consultation;`	Supprime une contrainte par son nom (section 7)

Changer le **type** d'une colonne s'écrit différemment selon le SGBD :

SGBD	Agrandir `nom_medicament` à 100 caractères
MySQL	`ALTER TABLE medicament MODIFY nom_medicament VARCHAR(100) NOT NULL;` (on réécrit toute la définition, `NOT NULL` compris)
PostgreSQL	`ALTER TABLE medicament ALTER COLUMN nom_medicament TYPE VARCHAR(100);`
🔶 Oracle	`ALTER TABLE medicament MODIFY (nom_medicament VARCHAR2(100));`

> 🪛 **Essayez** la syntaxe d'**un autre** SGBD que le vôtre : MySQL répond `ERROR 1064 ... near 'TYPE VARCHAR(100)'` ; PostgreSQL répond `ERREUR: erreur de syntaxe sur ou près de « MODIFY »`. C'est l'un des rares endroits où les SGBD ne s'entendent pas.

4.2 `DROP TABLE` et `TRUNCATE TABLE`

Instruction	Ce qu'elle supprime	Peut-on revenir en arrière ?
`DELETE FROM t;`	Toutes les **lignes** (la table reste)	Oui, dans une transaction (section 8)
`TRUNCATE TABLE t;`	Toutes les **lignes**, très vite ; remet aussi la numérotation automatique à zéro (section 9)	Non sur MySQL et Oracle (elle valide la transaction)
`DROP TABLE t;`	La **table entière** : structure, lignes, contraintes	Non

> 🪛 **Essayez** : `DROP TABLE service;` (des médecins y sont rattachés).
>
> - MySQL (MariaDB) : `ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails`
> - PostgreSQL : `ERREUR: n'a pas pu supprimer table service car d'autres objets en dépendent`, puis `ASTUCE : Utilisez DROP ... CASCADE pour supprimer aussi les objets dépendants.`
>
> On ne peut pas supprimer une table **parent** tant que des tables enfants la désignent. Pour tout supprimer, on part des tables enfants (ordre inverse de la création). N'utilisez pas `CASCADE` sans savoir ce qu'il emporte.

5. Modifier et supprimer des lignes : `UPDATE` et `DELETE`

```sql
UPDATE patient SET groupe_sanguin = 'O+' WHERE num_patient = 1;
UPDATE consultation SET tarif = tarif + 2000 WHERE date_consultation = DATE '2026-10-05';
DELETE FROM prescription WHERE num_consultation = 2 AND code_medicament = 'PARA500';

SELECT num_consultation, tarif FROM consultation ORDER BY num_consultation;
SELECT COUNT(*) AS nb_prescriptions FROM prescription;
```

```
 num_consultation | tarif
------------------+-------
                1 | 12000
                2 | 17000
                3 | 20000
(3 lignes)

 nb_prescriptions
------------------
                2
(1 ligne)
```

Morceau	Ce qu'il veut dire
`UPDATE patient SET groupe_sanguin = 'O+'`	Change la valeur d'une colonne…
`WHERE num_patient = 1`	… **seulement** pour les lignes qui respectent la condition
`SET tarif = tarif + 2000`	La nouvelle valeur peut se calculer à partir de l'ancienne : les deux consultations du 5 octobre passent de 10000 à 12000 et de 15000 à 17000
`DELETE FROM prescription WHERE ... AND ...`	Supprime les lignes qui respectent **les deux** conditions : une seule prescription ici
`ORDER BY num_consultation`	Trie le résultat (chapitre 4). Sans `ORDER BY`, l'ordre des lignes **n'est pas garanti** : après un `UPDATE`, PostgreSQL peut les afficher dans un autre ordre
`COUNT(*)`	Compte les lignes (chapitre 4)

> ⚠️ **Le `WHERE` oublié.** `UPDATE consultation SET tarif = 0;` met **toutes** les consultations à 0 ; `DELETE FROM patient;` tente de supprimer **tous** les patients. Le SGBD ne demande pas de confirmation. Deux réflexes : écrire d'abord le `SELECT` avec le même `WHERE` pour voir les lignes concernées, et travailler dans une **transaction** (section 8) pour pouvoir annuler.

> 🧨 **Essayez** : `DELETE FROM patient WHERE num_patient = 1;` (Awa Diop a une consultation).
>
> - MySQL : `ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails (... 'fk_consultation_patient' ...)`
> - PostgreSQL : `ERREUR: UPDATE ou DELETE sur la table « patient » viole la contrainte de clé étrangère « fk_consultation_patient » de la table « consultation »`
>
> Supprimer Awa laisserait une consultation sans patient : refusé. La section 7 montre comment choisir un autre comportement.

6. Insérer ou mettre à jour : `MERGE`

La pharmacie reçoit sa liste de médicaments : certains existent déjà (il faut mettre à jour leur forme), d'autres sont nouveaux (il faut les ajouter). C'est le rôle de `MERGE`, de la norme SQL :

```
MERGE INTO medicament m
USING (VALUES ('PARA500', 'Paracétamol 500 mg', 'gélule'),
              ('IBU400',  'Ibuprofène 400 mg',  'comprimé')) AS n (code_medicament,
         nom_medicament, forme)
ON m.code_medicament = n.code_medicament
WHEN MATCHED THEN UPDATE SET forme = n.forme
WHEN NOT MATCHED THEN INSERT (code_medicament, nom_medicament, forme)
                      VALUES (n.code_medicament, n.nom_medicament, n.forme);
SELECT * FROM medicament ORDER BY code_medicament;
```

```
 code_medicament |   nom_medicament    |  forme
-----------------+---------------------+----------
 AMOX250         | Amoxicilline 250 mg | sirop
 IBU400          | Ibuprofène 400 mg   | comprimé
 PARA500         | Paracétamol 500 mg  | gélule
(3 lignes)
```

Morceau	Ce qu'il veut dire
`MERGE INTO medicament m`	La table à mettre à jour, appelée `m` dans la suite
`USING (VALUES ...) AS n (...)`	Les nouvelles données, appelées `n`, avec le nom de leurs colonnes
`ON m.code_medicament = n.code_medicament`	Comment reconnaître qu'un médicament existe déjà
`WHEN MATCHED THEN UPDATE ...`	Il existe (PARA500) : on met sa forme à jour
`WHEN NOT MATCHED THEN INSERT ...`	Il n'existe pas (IBU400) : on l'ajoute

`MERGE` existe sur **PostgreSQL** (version 15 et plus) et sur **Oracle**. **MySQL** ne le connaît pas ; il propose une forme plus courte, qui donne le même résultat :

```
INSERT INTO medicament (code_medicament, nom_medicament, forme) VALUES ('PARA500', 'Paracétamol
        500 mg', 'gélule')
ON DUPLICATE KEY UPDATE forme = VALUES(forme);
INSERT INTO medicament (code_medicament, nom_medicament, forme) VALUES ('IBU400', 'Ibuprofène 400
        mg', 'comprimé')
ON DUPLICATE KEY UPDATE forme = VALUES(forme);
```

🔶 **Oracle** Oracle écrit la source avec `SELECT ... FROM dual` (une table spéciale d'une ligne), sans `AS` devant l'alias, et met la condition `ON` entre parenthèses :

```
MERGE INTO medicament m
USING (SELECT 'PARA500' AS code_medicament, 'Paracétamol 500 mg' AS nom_medicament, 'gélule'
         AS forme FROM dual
       UNION ALL
       SELECT 'IBU400', 'Ibuprofène 400 mg', 'comprimé' FROM dual) n
ON (m.code_medicament = n.code_medicament)
WHEN MATCHED THEN UPDATE SET m.forme = n.forme
WHEN NOT MATCHED THEN INSERT (code_medicament, nom_medicament, forme)
                      VALUES (n.code_medicament, n.nom_medicament, n.forme);
```

7. Que faire des enfants quand on supprime le parent ?

Par défaut, une clé étrangère **interdit** de supprimer une ligne parent qui a des enfants (section 5). On peut choisir un autre comportement en ajoutant une clause à la clé étrangère :

Clause	Quand on supprime le parent…	Exemple pertinent
(aucune) ou `ON DELETE RESTRICT` / `NO ACTION`	… c'est **refusé** s'il a des enfants	Un patient qui a des consultations
`ON DELETE CASCADE`	… ses enfants sont **supprimés aussi**	Supprimer une consultation supprime ses prescriptions
`ON DELETE SET NULL`	… la clé étrangère des enfants devient `NULL` (elle doit être facultative)	Supprimer une assurance : les patients n'ont plus d'assurance

Une prescription n'a pas de sens sans sa consultation : on remplace la clé étrangère par une version `CASCADE`.

```
ALTER TABLE prescription DROP CONSTRAINT fk_prescription_consultation;
ALTER TABLE prescription ADD CONSTRAINT fk_prescription_consultation
    FOREIGN KEY (num_consultation) REFERENCES consultation (num_consultation) ON DELETE CASCADE;
DELETE FROM consultation WHERE num_consultation = 2;
SELECT num_consultation, code_medicament FROM prescription ORDER BY num_consultation;
```

```
 num_consultation | code_medicament
------------------+-----------------
                1 | PARA500
(1 ligne)
```

La consultation n°2 a disparu, et avec elle ses deux prescriptions (sur la base de la section 3, sans les modifications de la section 5).

> ⚠️ `CASCADE` est puissant et dangereux : un `DELETE` peut effacer des centaines de lignes dans d'autres tables. Ne le mettez que là où l'enfant **n'a aucun sens** sans son parent. Pour les patients et les consultations (dossiers médicaux), on garde le refus.

> 🔵 **Standard : les anciennes versions de MySQL** Avant MySQL 8.0.19, `DROP CONSTRAINT` n'existe pas pour une clé étrangère : on écrit `ALTER TABLE prescription DROP FOREIGN KEY fk_prescription_consultation;` .

8. Les transactions

8.1 Tout ou rien

Une **transaction** est un groupe de modifications qui doivent réussir **toutes ensemble**, ou pas du tout. Exemple : enregistrer une consultation **et** ses prescriptions ; si une prescription échoue, on ne veut pas d'une consultation à moitié enregistrée.

Instruction	Ce qu'elle fait
`START TRANSACTION;`	Commence une transaction (on peut aussi écrire `BEGIN;` sur MySQL et PostgreSQL)
`COMMIT;`	**Valide** : les modifications deviennent définitives et visibles par tous
`ROLLBACK;`	**Annule** toutes les modifications depuis le début de la transaction
`SAVEPOINT nom;`	Pose un point de repère au milieu de la transaction
`ROLLBACK TO SAVEPOINT nom;`	Annule seulement ce qui a été fait **depuis** ce repère

```sql
START TRANSACTION;
UPDATE consultation SET tarif = 0 WHERE num_consultation = 3;
DELETE FROM prescription;
SELECT COUNT(*) AS prescriptions_pendant FROM prescription;
ROLLBACK;
SELECT COUNT(*) AS prescriptions_apres FROM prescription;
SELECT tarif FROM consultation WHERE num_consultation = 3;

START TRANSACTION;
INSERT INTO consultation VALUES (4, DATE '2026-10-20', 'Vaccination', 5000, 3, 4);
SAVEPOINT apres_consultation;
INSERT INTO prescription VALUES (4, 'PARA500', '1/2 comprimé si fièvre', 2);
ROLLBACK TO SAVEPOINT apres_consultation;
COMMIT;
SELECT num_consultation, motif FROM consultation ORDER BY num_consultation;
SELECT COUNT(*) AS prescriptions_consultation_4 FROM prescription WHERE num_consultation = 4;
```

Sur la base de la section 3 : pendant la première transaction, il reste `0` prescription ; après le `ROLLBACK`, on en retrouve `3`, et le tarif de la consultation n°3 vaut toujours `20000`. Dans la seconde, la consultation n°4 (`Vaccination`) est validée, mais sa prescription a été annulée par le `ROLLBACK TO SAVEPOINT` : `0` prescription pour la consultation 4.

8.2 Une erreur au milieu d'une transaction

```sql
START TRANSACTION;
INSERT INTO consultation VALUES (5, DATE '2026-10-21', 'Contrôle', 5000, 1, 1);
INSERT INTO consultation VALUES (6, DATE '2026-10-21', 'Contrôle', 5000, 99, 1);
INSERT INTO consultation VALUES (7, DATE '2026-10-21', 'Contrôle', 5000, 2, 1);
COMMIT;
```

La deuxième ligne échoue (le patient 99 n'existe pas). Ensuite, les SGBD ne réagissent **pas** de la même façon :

SGBD	Ce qui se passe	Consultations enregistrées
MySQL	Seule l'instruction fautive est refusée ; la transaction continue ; le `COMMIT` valide les autres	5 et 7
PostgreSQL	Toute la transaction est **bloquée** : `ERREUR: la transaction est annulée, les commandes sont ignorées jusqu'à la fin du bloc de la transaction` ; le `COMMIT` se transforme en annulation	aucune

La règle qui marche **partout** : **dès qu'une erreur survient dans une transaction, faites** `ROLLBACK`, corrigez, et recommencez la transaction.

🟢 **Débutant : et sans `START TRANSACTION` ?** MySQL et PostgreSQL sont, par défaut, en mode **validation automatique** (*autocommit*) : chaque instruction est une petite transaction validée aussitôt. C'est pourquoi tous vos `INSERT` précédents ont été enregistrés sans `COMMIT`. 🔶 Oracle fonctionne à l'inverse : une transaction commence toute seule à la première modification, et **rien n'est définitif avant `COMMIT`** (selon les réglages de votre outil). Sur Oracle, pensez à `COMMIT` avant de fermer FreeSQL.

🔵 **Standard : les instructions LDD et les transactions** Sur MySQL et Oracle, un `CREATE`, `ALTER`, `DROP` ou `TRUNCATE` **valide** automatiquement la transaction en cours : `START TRANSACTION; CREATE TABLE essai_ddl (n INT); ROLLBACK;` laisse la table `essai_ddl` créée. Sur PostgreSQL, le `ROLLBACK` la fait disparaître. Ne mélangez pas création de tables et modifications de données dans une même transaction.

🟣 **Expert : ACID** Les transactions des SGBD relationnels garantissent quatre propriétés, résumées par le sigle **ACID** : **A**tomicité (tout ou rien), **C**ohérence (les contraintes sont respectées à la fin), **I**solation (deux transactions simultanées ne se voient pas à moitié), **D**urabilité (après `COMMIT`, une panne ne fait rien perdre). Vous reverrez l'isolation et la concurrence en BDD2.

9. Numéroter automatiquement : identité et séquences

Choisir soi-même le prochain numéro (`num_patient = 4`) devient vite risqué : deux secrétaires peuvent prendre le même au même moment. Le SGBD sait **numéroter tout seul**.

SGBD	Colonne numérotée automatiquement
MySQL	`num_lot INT AUTO_INCREMENT PRIMARY KEY`
PostgreSQL et 🔶 Oracle (12c et plus)	`num_lot INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY` (norme SQL)

Exemple avec les lots de médicaments de la pharmacie (version PostgreSQL ; pour MySQL, remplacez la deuxième ligne par `num_lot INT AUTO_INCREMENT PRIMARY KEY,`) :

```
CREATE TABLE medicament_stock (
    num_lot   INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    code      VARCHAR(10) NOT NULL,
    quantite  INT NOT NULL
);
INSERT INTO medicament_stock (code, quantite) VALUES ('PARA500', 200);
INSERT INTO medicament_stock (code, quantite) VALUES ('AMOX250', 50), ('PARA500', 100);
DELETE FROM medicament_stock WHERE num_lot = 3;
INSERT INTO medicament_stock (code, quantite) VALUES ('IBU400', 80);
SELECT * FROM medicament_stock;
```

```
 num_lot |  code   | quantite
---------+---------+----------
       1 | PARA500 |      200
       2 | AMOX250 |       50
       4 | IBU400  |       80
(3 lignes)
```

Morceau	Ce qu'il veut dire
`INSERT INTO medicament_stock (code, quantite)`	On ne donne **pas** `num_lot` : le SGBD le calcule (1, 2, 3…)
Le lot n°3 supprimé, puis `IBU400` reçoit le n°**4**	Un numéro attribué n'est **jamais réutilisé**, même s'il a été supprimé. Les « trous » sont normaux
`TRUNCATE TABLE medicament_stock;`	Vide la table **et** remet le compteur à 1 sur MySQL ; sur PostgreSQL, il faut écrire `TRUNCATE TABLE medicament_stock RESTART IDENTITY;`

La séquence est un compteur indépendant de toute table, utile pour numéroter plusieurs tables ou des factures (PostgreSQL et Oracle ; MySQL n'en a pas) :

```sql
CREATE SEQUENCE seq_facture START WITH 1000 INCREMENT BY 1;
SELECT nextval('seq_facture');
```

Sur PostgreSQL, le premier appel donne `1000`, le suivant `1001`. 🔶 Oracle écrit `SELECT seq_facture.NEXTVAL FROM dual;`.

Exercices

Travaillez dans une base (ou un schéma) `exercices_ch3`, distincte de `clinique`.

- ★ **Exercice 1 : choisir les types.** Donnez le type SQL (et la taille) de chaque colonne : a) le numéro d'un bus de la ligne 8 ; b) le prix d'un billet en FCFA ; c) la note d'un étudiant sur 20, avec une décimale ; d) un numéro de téléphone ; e) la date et l'heure de départ d'un vol ; f) le code IATA d'un aéroport (`DSS`) ; g) un texte de présentation de 2000 caractères ; h) « l'étudiant est-il boursier ? ». (Section 1.)
- ★ **Exercice 2 : la boutique de quartier.** Une boutique de Pikine veut deux tables : `categorie` (code de 3 lettres, libellé unique) et `produit` (numéro, nom obligatoire, prix en FCFA strictement positif, stock positif ou nul valant 0 par défaut, catégorie obligatoire). Écrivez les deux `CREATE TABLE` avec des contraintes **nommées**, puis insérez 3 catégories et 6 produits. Provoquez une erreur de chaque contrainte et notez les messages. (Sections 2 et 3.)
- ★★ **Exercice 3 : mises à jour.** Sur la boutique : a) augmentez de 10 % le prix des produits d'une catégorie ; b) mettez le stock à 0 pour les produits dont le stock est inférieur à 3 ; c) supprimez les produits d'une catégorie ; d) ajoutez une colonne `date_ajout` avec la date du jour par défaut (cherchez `CURRENT_DATE`). Avant chaque `UPDATE` ou `DELETE`, écrivez le `SELECT` qui montre les lignes concernées. (Sections 4 et 5.)

- ★★ **Exercice 4 : transaction de vente.** Ajoutez une table `vente` (numéro automatique, date, produit, quantité). Écrivez une transaction qui enregistre une vente de 4 unités **et** diminue le stock du produit ; puis rejouez-la pour un produit dont le stock est insuffisant : ajoutez un `CHECK (stock >= 0)`, observez l'erreur, et terminez par le bon ordre (`ROLLBACK`). Vérifiez qu'aucune vente n'a été enregistrée. (Sections 8 et 9.)
- ★★★ **Exercice 5 : la bibliothèque, du MLD au script.** Reprenez le MLD de l'exercice 4 du chapitre 2 (bibliothèque universitaire) et écrivez le script complet : tables dans le bon ordre, toutes les contraintes, une suppression en cascade bien choisie (justifiez), un jeu de données (au moins 3 étudiants, 4 livres, 6 exemplaires, 5 emprunts dont 2 non rendus). Le script doit pouvoir être **rejoué** : commencez-le par les `DROP TABLE` dans l'ordre inverse. (Toutes les sections.)

Atelier 3 : la base de la clinique

Vous écrivez le script complet de la base de la clinique Santé Plus à partir de **votre** MLD de l'atelier 2 (dix tables), avec toutes les contraintes, un jeu de données réaliste, des mises à jour et une transaction. L'énoncé de l'atelier vous est fourni séparément.

A retenir

- **LDD** : `CREATE TABLE` , `ALTER TABLE` (`ADD` , `DROP COLUMN` , `ADD CONSTRAINT`), `DROP TABLE` , `TRUNCATE TABLE` . **LMD** : `INSERT` , `UPDATE` , `DELETE` , `MERGE` . **LCT** : `COMMIT` , `ROLLBACK` , `SAVEPOINT` .
- Contraintes : `PRIMARY KEY` (entité), `FOREIGN KEY ... REFERENCES` (référentielle), `NOT NULL` , `UNIQUE` , `CHECK` , `DEFAULT` (domaine). Nommez-les avec `CONSTRAINT` .
- On crée les tables **parents d'abord** ; on les supprime **enfants d'abord**.
- `INSERT` avec la liste des colonnes ; une apostrophe dans un texte se double.
- `UPDATE` et `DELETE` **toujours** avec un `WHERE` vérifié par un `SELECT` .
- Clé étrangère à la suppression du parent : refus (par défaut), `CASCADE` , `SET NULL` .
- Transaction : `START TRANSACTION` ... `COMMIT` ou `ROLLBACK` . Après une erreur : `ROLLBACK` .
- Numérotation : `AUTO_INCREMENT` (MySQL), `GENERATED BY DEFAULT AS IDENTITY` (PostgreSQL, Oracle), séquences ; un numéro n'est jamais réutilisé.
- Sans `ORDER BY` , l'ordre des lignes n'est pas garanti.

Erreurs fréquentes

Message (MySQL / PostgreSQL)	Ce que ça veut dire	Solution
`Cannot add or update a child row: a foreign key constraint fails` / `viole la contrainte de clé étrangère`	La valeur de la clé étrangère n'existe pas dans la table parent	Insérer d'abord le parent ; vérifier la valeur
`Cannot delete or update a parent row` / `UPDATE ou DELETE sur la table ... viole la contrainte de clé étrangère`	On supprime un parent qui a des enfants	Supprimer d'abord les enfants, ou prévoir `ON DELETE CASCADE` / `SET NULL`

Message (MySQL / PostgreSQL)	Ce que ça veut dire	Solution
`Can't create table ... (errno: 150 ...)` / `la relation « x » n'existe pas` (au `CREATE TABLE`)	La table parent n'existe pas encore, ou les types ne correspondent pas	Créer les parents d'abord ; même type pour la clé étrangère et la clé primaire
`CONSTRAINT ... failed` / `viole la contrainte de vérification`	Une valeur ne respecte pas un `CHECK`	Corriger la valeur, ou la règle
`Duplicate entry ... for key` / `rompt la contrainte unique`	Valeur en double sur une clé primaire ou une colonne `UNIQUE`	Autre valeur
`n'a pas pu supprimer table ... car d'autres objets en dépendent`	`DROP` d'une table parent	Supprimer les tables enfants d'abord
`la transaction est annulée, les commandes sont ignorées` (PostgreSQL)	Une erreur a eu lieu plus tôt dans la transaction	`ROLLBACK;`, corriger, recommencer
Toutes les lignes modifiées ou supprimées	`WHERE` oublié	Transaction + `ROLLBACK` ; toujours tester le `WHERE` avec un `SELECT`
`... near 'TYPE ...'` / `... près de « MODIFY »`	Syntaxe d'`ALTER` d'un autre SGBD	Voir le tableau de la section 4.1

Pour aller plus loin

- MySQL, *CREATE TABLE* et *FOREIGN KEY* : https://dev.mysql.com/doc/refman/8.4/en/create-table.html
- PostgreSQL en français, *Définition des données* et *Contraintes* : https://docs.postgresql.fr/16/ddl.html
- Oracle, *SQL Language Reference* (`CREATE TABLE`, `MERGE`) : https://docs.oracle.com/en/database/oracle/oracle-database/
- Oracle Academy, *Applied Database Systems*, section 2 (support du syllabus).